

附录 A 源代码

verbatim

项目目录结构

```
.
├── BatteryModel/
│   ├── analysis/
│   │   ├── advanced_analysis.py
│   │   ├── common.py
│   │   ├── scenario_analysis.py
│   │   └── sensitivity.py
│   ├── models/
│   │   ├── behavior.py
│   │   └── state.py
│   ├── params/
│   │   ├── aging_params.py
│   │   ├── electrical_params.py
│   │   ├── load_params.py
│   │   ├── resistance_params.py
│   │   ├── soc_params.py
│   │   └── thermal_params.py
│   ├── physics/
│   │   ├── electrical.py
│   │   ├── load.py
│   │   ├── resistance.py
│   │   ├── soc.py
│   │   ├── soh.py
│   │   ├── thermal.py
│   │   └── tte.py
│   └── simulation/
│       ├── config.py
│       └── simulator.py
├── appendix.tex
└── generate_all_figures_1.py
```

A.1 Analysis Module

A.1.1 advanced_analysis.py

```
1  """Advanced analyses for the SOC-SOH-2RC-thermal-TTE battery model."""
2  from __future__ import annotations
3
4  from copy import deepcopy
5  from pathlib import Path
6  from time import perf_counter
7  import math
8
9  import matplotlib
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14
15 from BatteryModel.analysis.common import (
16     ModelParameters,
17     default_parameters,
18     ensure_output_dir,
19     run_scenario_case,
20     save_dataframe,
21 )
22 from BatteryModel.analysis.scenario_analysis import (
23     SCENARIO_LIBRARY,
24     behavior_from_scenario,
25     get_scenario,
26 )
27 from BatteryModel.models.behavior import PiecewiseRandomBehavior
28 from BatteryModel.models.state import BatteryState
29 from BatteryModel.physics.load import calc_load_power
30 from BatteryModel.physics.thermal import low_temp_factor
31 from BatteryModel.physics.soh import soh_derivative
32 from BatteryModel.simulation.config import SimulationConfig
33 from BatteryModel.simulation.simulator import simulate_case
34
35 PLOT_DPI = 300
36
37
38 def _save_fig(fig: plt.Figure, path: Path) -> None:
39     fig.tight_layout()
40     fig.savefig(path, dpi=PLOT_DPI, bbox_inches="tight")
41     plt.close(fig)
42
43
44 def _base_config(dt_s: float = 5.0, record_interval_s: float = 30.0) ->
45     SimulationConfig:
46     return SimulationConfig(
47         dt_s=dt_s,
48         max_time_s=24 * 3600,
49         record_interval_s=record_interval_s,
50         soc_min=0.03,
51         voltage_cutoff_v=3.0,
52     )
```

```

53
54 def initial_soc_analysis(output_dir: str | Path, quick: bool = False) -> pd
    .DataFrame:
55     output = ensure_output_dir(output_dir) / "01_initial_soc"
56     output.mkdir(parents=True, exist_ok=True)
57     soc_values = [0.2, 0.4, 0.6, 0.8, 1.0]
58     scenarios = ["office", "video", "navigation", "game"]
59     rows = []
60     for scenario_name in scenarios:
61         for soc in soc_values:
62             result, scenario, _ = run_scenario_case(
63                 scenario_name,
64                 initial_soc=soc,
65                 config=_base_config(dt_s=10.0 if quick else 5.0),
66             )
67             rows.append({
68                 "scenario": scenario["name"],
69                 "initial_soc": soc,
70                 "tte_h": result.tte_hours,
71                 "stop_reason": result.stop_reason,
72                 "final_soc": result.final_state.soc,
73                 "final_voltage_v": result.final_state.voltage,
74             })
75     df = pd.DataFrame(rows)
76     save_dataframe(df, output / "initial_soc_tte.csv")
77
78     fig, ax = plt.subplots(figsize=(8.2, 5.2))
79     for name, group in df.groupby("scenario"):
80         ax.plot(group["initial_soc"] * 100, group["tte_h"], marker="o",
81 label=name)
82     ax.set_xlabel("Initial SOC (%)")
83     ax.set_ylabel("Actual TTE (h)")
84     ax.set_title("Initial SOC—TTE relationship")
85     ax.grid(alpha=0.3)
86     ax.legend()
87     _save_fig(fig, output / "initial_soc_tte.png")
88
89     pivot = df.pivot(index="initial_soc", columns="scenario", values="tte_h
90 ")
91     fig, ax = plt.subplots(figsize=(8.2, 5.2))
92     pivot.plot(kind="bar", ax=ax)
93     ax.set_xlabel("Initial SOC")
94     ax.set_ylabel("Actual TTE (h)")
95     ax.set_title("Scenario comparison at identical initial SOC")
96     ax.grid(axis="y", alpha=0.3)
97     _save_fig(fig, output / "initial_soc_scenario_comparison.png")
98     return df
99
100 def temperature_analysis(output_dir: str | Path, quick: bool = False) -> pd
    .DataFrame:
101     output = ensure_output_dir(output_dir) / "02_temperature"
102     output.mkdir(parents=True, exist_ok=True)
103     temperatures = [-10, 0, 10, 25, 40]
104     scenarios = ["office", "video", "navigation", "game"]
105     rows = []
106     detailed = {}
107     for scenario_name in scenarios:

```

```

107     for temp in temperatures:
108         result, scenario, params = run_scenario_case(
109             scenario_name,
110             ambient_temp=temp,
111             config=_base_config(dt_s=10.0 if quick else 5.0),
112         )
113         initial_soh_loss_rate = float("nan")
114         if result.history:
115             first = result.history[0]
116             rate_state = BatteryState(
117                 soc=first["soc"], soh=first["soh"], temp=first["temp"],
118                 voltage=first["voltage"], current=first["current"],
119                 v1=first["v1"], v2=first["v2"],
120             )
121             initial_soh_loss_rate = -soh_derivative(
122                 rate_state, 3.45, params.aging
123             ) * 3600.0
124             rows.append({
125                 "scenario": scenario["name"],
126                 "ambient_temp_c": temp,
127                 "tte_h": result.tte_hours,
128                 "stop_reason": result.stop_reason,
129                 "final_temp_c": result.final_state.temp,
130                 "final_voltage_v": result.final_state.voltage,
131                 "final_soc": result.final_state.soc,
132                 "initial_soh_loss_rate_per_hour": initial_soh_loss_rate,
133             })
134             if scenario_name == "game":
135                 detailed[temp] = pd.DataFrame(result.history)
136         df = pd.DataFrame(rows)
137         baseline_rates = (
138             df[df["ambient_temp_c"] == 25]
139             .set_index("scenario")["initial_soh_loss_rate_per_hour"]
140             .to_dict()
141         )
142         df["relative_aging_rate_vs_25c"] = [
143             row.initial_soh_loss_rate_per_hour / max(baseline_rates.get(row.
144             scenario, np.nan), 1e-30)
145             for row in df.itertuples()
146         ]
147         save_dataframe(df, output / "temperature_tte.csv")
148
149         fig, ax = plt.subplots(figsize=(8.2, 5.2))
150         for name, group in df.groupby("scenario"):
151             ax.plot(group["ambient_temp_c"], group["tte_h"], marker="o", label=
152             name)
153             ax.set_xlabel("Ambient temperature (°C)")
154             ax.set_ylabel("Actual TTE (h)")
155             ax.set_title("Environmental temperature effect on TTE")
156             ax.grid(alpha=0.3)
157             ax.legend()
158             _save_fig(fig, output / "temperature_tte.png")
159
160         pivot = df.pivot(index="scenario", columns="ambient_temp_c", values="
161         tte_h")
162         fig, ax = plt.subplots(figsize=(8.4, 4.8))
163         im = ax.imshow(pivot.values, aspect="auto", cmap="viridis")
164         ax.set_xticks(range(len(pivot.columns)), pivot.columns)

```

```

162 ax.set_yticks(range(len(pivot.index)), pivot.index)
163 ax.set_xlabel("Ambient temperature (°C)")
164 ax.set_title("Scenario-temperature TTE heatmap")
165 fig.colorbar(im, ax=ax, label="TTE (h)")
166 _save_fig(fig, output / "temperature_scenario_heatmap.png")
167
168 for variable, label, filename in [
169     ("voltage", "Terminal voltage (V)", "game_voltage_by_temperature.
170 png"),
171     ("temp", "Battery temperature (°C)", "game_temperature_trajectories
172 .png"),
173     ("soc", "SOC", "game_soc_by_temperature.png"),
174 ]:
175     fig, ax = plt.subplots(figsize=(8.2, 5.2))
176     for temp, hist in detailed.items():
177         ax.plot(hist["time"] / 3600, hist[variable], label=f"{temp}°C")
178     ax.set_xlabel("Time (h)")
179     ax.set_ylabel(label)
180     ax.set_title(f"Mobile game: {label} trajectories")
181     ax.grid(alpha=0.3)
182     ax.legend()
183     _save_fig(fig, output / filename)
184
185 fig, ax = plt.subplots(figsize=(8.2, 5.2))
186 for name, group in df.groupby("scenario"):
187     ax.plot(group["ambient_temp_c"], group["relative_aging_rate_vs_25c"
188 ], marker="o", label=name)
189 ax.set_yscale("log")
190 ax.set_xlabel("Ambient temperature (°C)")
191 ax.set_ylabel("Relative initial aging rate (25°C = 1)")
192 ax.set_title("Temperature acceleration of SOH degradation")
193 ax.grid(alpha=0.3)
194 ax.legend()
195 _save_fig(fig, output / "temperature_relative_aging_rate.png")
196 return df
197
198 def soh_analysis(output_dir: str | Path, quick: bool = False) -> pd.
199 DataFrame:
200     output = ensure_output_dir(output_dir) / "03_soh"
201     output.mkdir(parents=True, exist_ok=True)
202     soh_values = [1.0, 0.9, 0.8, 0.7]
203     rows = []
204     histories = {}
205     for soh in soh_values:
206         result, scenario, _ = run_scenario_case(
207             "game",
208             initial_soh=soh,
209             config=_base_config(dt_s=10.0 if quick else 5.0),
210         )
211         rows.append({
212             "scenario": scenario["name"],
213             "initial_soh": soh,
214             "tte_h": result.tte_hours,
215             "stop_reason": result.stop_reason,
216             "final_voltage_v": result.final_state.voltage,
217             "final_soc": result.final_state.soc,
218         })

```

```

216     histories[soh] = pd.DataFrame(result.history)
217     df = pd.DataFrame(rows)
218     tte_new = float(df.loc[df["initial_soh"] == 1.0, "tte_h"].iloc[0])
219     df["tte_decline_pct"] = (tte_new - df["tte_h"]) / tte_new * 100
220     save_dataframe(df, output / "soh_tte.csv")
221
222     fig, ax = plt.subplots(figsize=(8.2, 5.2))
223     ax.plot(df["initial_soh"] * 100, df["tte_h"], marker="o")
224     ax.set_xlabel("Initial SOH (%)")
225     ax.set_ylabel("Actual TTE (h)")
226     ax.set_title("SOH effect on mobile-game endurance")
227     ax.grid(alpha=0.3)
228     _save_fig(fig, output / "soh_tte.png")
229
230     fig, ax = plt.subplots(figsize=(8.2, 5.2))
231     for soh, hist in histories.items():
232         ax.plot(hist["time"] / 3600, hist["voltage"], label=f"SOH={soh:.1f}")
233     ax.axhline(3.0, linestyle="--", linewidth=1, label="Voltage cutoff")
234     ax.set_xlabel("Time (h)")
235     ax.set_ylabel("Terminal voltage (V)")
236     ax.set_title("Voltage trajectories under battery aging")
237     ax.grid(alpha=0.3)
238     ax.legend()
239     _save_fig(fig, output / "soh_voltage_trajectories.png")
240     return df
241
242
243 def _analytic_baseline_tte_h(
244     scenario: dict,
245     initial_soc: float,
246     initial_soh: float,
247     ambient_temp: float,
248     params: ModelParameters,
249     soc_min: float = 0.03,
250 ) -> float:
251     user = behavior_from_scenario(scenario)
252     power = calc_load_power(user, params.load)["total"]
253     usable_fraction = max(initial_soc - soc_min, 0.0)
254     capacity_factor = max(low_temp_factor(ambient_temp, params.thermal),
255                            0.05)
256     energy_wh = usable_fraction * initial_soh * params.electrical.__dict__.
257     get("nominal_voltage", 3.7)
258     energy_wh *= 3.45 * capacity_factor
259     return energy_wh / max(power, 1e-9)
260
261 def model_ablation_analysis(output_dir: str | Path, quick: bool = False) ->
262     pd.DataFrame:
263     output = ensure_output_dir(output_dir) / "04_ablation"
264     output.mkdir(parents=True, exist_ok=True)
265     cases = [
266         ("Standby", "standby", 25.0, 1.0),
267         ("Game", "game", 25.0, 1.0),
268         ("Low-temperature game", "game", 0.0, 1.0),
269         ("Aged game", "game", 25.0, 0.8),
270     ]
271     rows = []

```

```

270     for case_name, scenario_name, ambient, soh in cases:
271         scenario = get_scenario(scenario_name)
272         base_params = default_parameters()
273         rows.append({
274             "case": case_name,
275             "model": "Baseline energy/power",
276             "tte_h": _analytic_baseline_tte_h(scenario, 1.0, 1.0, 25.0,
base_params),
277             "stop_reason": "analytic estimate",
278         })
279
280         # Model A: SOC dynamics only; no voltage, thermal, or aging
correction.
281         neutral = deepcopy(base_params)
282         neutral.thermal.T_env = 25.0
283         cfg_a = _base_config(dt_s=10.0 if quick else 5.0)
284         cfg_a.use_2rc = False
285         cfg_a.use_thermal = False
286         cfg_a.use_soh = False
287         cfg_a.use_dynamic_resistance = False
288         cfg_a.stop_on_voltage = False
289         result, _, _ = run_scenario_case(
290             scenario_name, initial_soh=1.0, ambient_temp=25.0,
291             config=cfg_a, params=neutral,
292         )
293         rows.append({"case": case_name, "model": "Model A: SOC", "tte_h":
result.tte_hours, "stop_reason": result.stop_reason})
294
295         # Model B: SOC + 2RC; condition-specific thermal and SOH effects
omitted.
296         cfg_b = _base_config(dt_s=10.0 if quick else 5.0)
297         cfg_b.use_thermal = False
298         cfg_b.use_soh = False
299         result, _, _ = run_scenario_case(
300             scenario_name, initial_soh=1.0, ambient_temp=25.0,
301             config=cfg_b, params=neutral,
302         )
303         rows.append({"case": case_name, "model": "Model B: SOC + 2RC", "
tte_h": result.tte_hours, "stop_reason": result.stop_reason})
304
305         # Model C: add thermal effects, but keep a new battery.
306         cfg_c = _base_config(dt_s=10.0 if quick else 5.0)
307         cfg_c.use_soh = False
308         result, _, _ = run_scenario_case(
309             scenario_name, initial_soh=1.0, ambient_temp=ambient,
310             config=cfg_c,
311         )
312         rows.append({"case": case_name, "model": "Model C: + thermal", "
tte_h": result.tte_hours, "stop_reason": result.stop_reason})
313
314         # Full model: thermal + SOH + dynamic resistance/aging.
315         result, _, _ = run_scenario_case(
316             scenario_name, initial_soh=soh, ambient_temp=ambient,
317             config=_base_config(dt_s=10.0 if quick else 5.0),
318         )
319         rows.append({"case": case_name, "model": "Full model: + SOH", "
tte_h": result.tte_hours, "stop_reason": result.stop_reason})
320

```

```

321 df = pd.DataFrame(rows)
322 full = df[df["model"] == "Full model: + SOH"].set_index("case")["tte_h"
]
323 df["relative_error_to_full_pct"] = [
324     (row.tte_h - full.loc[row.case]) / full.loc[row.case] * 100
325     for row in df.itertuples()
326 ]
327 save_dataframe(df, output / "model_ablation.csv")
328
329 pivot = df.pivot(index="case", columns="model", values="tte_h")
330
331 fig, ax = plt.subplots(figsize=(10.5, 5.8))
332 pivot.plot(kind="bar", ax=ax)
333 ax.set_xlabel("")
334 ax.set_ylabel("TTE (h)")
335 ax.set_title("Incremental model / ablation comparison")
336
337 ax.set_xticklabels(ax.get_xticklabels(), rotation=0, ha="center")
338 ax.grid(axis="y", alpha=0.3)
339 ax.legend(fontsize=8)
340 _save_fig(fig, output / "model_ablation.png")
341 return df
342
343
344 def stop_reason_analysis(output_dir: str | Path, quick: bool = False) -> pd
.DataFrame:
345     output = ensure_output_dir(output_dir) / "05_stop_reason"
346     output.mkdir(parents=True, exist_ok=True)
347     temperatures = [-10, 0, 10, 25, 40, 55, 58]
348     soh_values = [0.7, 0.8, 0.9, 1.0]
349     rows = []
350     for temp in temperatures:
351         for soh in soh_values:
352             result, scenario, _ = run_scenario_case(
353                 "game", ambient_temp=temp, initial_soh=soh,
354                 config=_base_config(dt_s=10.0 if quick else 5.0),
355             )
356             rows.append({
357                 "scenario": scenario["name"],
358                 "ambient_temp_c": temp,
359                 "initial_soh": soh,
360                 "stop_reason": result.stop_reason,
361                 "tte_h": result.tte_hours,
362                 "final_soc": result.final_state.soc,
363                 "final_voltage_v": result.final_state.voltage,
364                 "final_temp_c": result.final_state.temp,
365             })
366     df = pd.DataFrame(rows)
367     save_dataframe(df, output / "stop_reason_table.csv")
368
369     reason_order = ["SOC threshold", "voltage cutoff", "thermal protection"
, "maximum simulation time"]
370     reason_to_code = {name: i for i, name in enumerate(reason_order)}
371     pivot = df.pivot(index="initial_soh", columns="ambient_temp_c", values=
"stop_reason")
372     coded = pivot.map(lambda value: reason_to_code.get(value, len(
reason_order)))
373     fig, ax = plt.subplots(figsize=(8.4, 4.8))

```



```

374     im = ax.imshow(coded.values, aspect="auto", cmap="tab10", vmin=0, vmax=
max(len(reason_order) - 1, 1))
375     ax.set_xticks(range(len(pivot.columns)), pivot.columns)
376     ax.set_yticks(range(len(pivot.index)), [f"{x:.1f}" for x in pivot.index
])
377     ax.set_xlabel("Ambient temperature (°C)")
378     ax.set_ylabel("Initial SOH")
379     ax.set_title("TTE termination mechanism regions (mobile game)")
380     handles = [plt.Line2D([0], [0], marker="s", linestyle="", markersize=8,
markerfacecolor=im.cmap(im.norm(i)),
381                  markeredgecolor="none", label=name)
for i, name in enumerate(reason_order) if name in set(df["
382 stop_reason"])]
383     ax.legend(handles=handles, loc="upper center", bbox_to_anchor=(0.5,
-0.16), ncol=2)
384     _save_fig(fig, output / "stop_reason_map.png")
385     return df
386
387
388 def _interaction_grid(
389     pair_name: str,
390     x_name: str,
391     x_values: list[float],
392     y_name: str,
393     y_values: list[float],
394     output: Path,
395     quick: bool,
396 ) -> pd.DataFrame:
397     rows = []
398     for y in y_values:
399         for x in x_values:
400             overrides = {}
401             ambient = 25.0
402             soh = 1.0
403             if x_name == "ambient_temp_c":
404                 ambient = x
405             elif x_name == "initial_soh":
406                 soh = x
407             else:
408                 overrides[x_name] = x
409             if y_name == "ambient_temp_c":
410                 ambient = y
411             elif y_name == "initial_soh":
412                 soh = y
413             else:
414                 overrides[y_name] = y
415             result, _, _ = run_scenario_case(
416                 "game",
417                 ambient_temp=ambient,
418                 initial_soh=soh,
419                 behavior_overrides=overrides,
420                 config=_base_config(dt_s=15.0 if quick else 10.0,
record_interval_s=120.0),
421             )
422             rows.append({x_name: x, y_name: y, "tte_h": result.tte_hours, "
stop_reason": result.stop_reason})
423     df = pd.DataFrame(rows)
424     save_dataframe(df, output / f"{pair_name}.csv")

```

```

425     pivot = df.pivot(index=y_name, columns=x_name, values="tte_h")
426     x_grid, y_grid = np.meshgrid(pivot.columns.astype(float), pivot.index.
427     astype(float))
428     z = pivot.values
429
430     fig, ax = plt.subplots(figsize=(7.6, 5.2))
431     im = ax.imshow(z, origin="lower", aspect="auto", cmap="viridis",
432     extent=[x_grid.min(), x_grid.max(), y_grid.min(), y_grid
433     .max()])
434     ax.set_xlabel(x_name)
435     ax.set_ylabel(y_name)
436     ax.set_title(f"{pair_name}: TTE heatmap")
437     fig.colorbar(im, ax=ax, label="TTE (h)")
438     _save_fig(fig, output / f"{pair_name}_heatmap.png")
439
440     fig, ax = plt.subplots(figsize=(7.6, 5.2))
441     contour = ax.contourf(x_grid, y_grid, z, levels=12, cmap="viridis")
442     ax.set_xlabel(x_name)
443     ax.set_ylabel(y_name)
444     ax.set_title(f"{pair_name}: TTE contours")
445     fig.colorbar(contour, ax=ax, label="TTE (h)")
446     _save_fig(fig, output / f"{pair_name}_contour.png")
447
448     fig = plt.figure(figsize=(8.0, 5.8))
449     ax3d = fig.add_subplot(111, projection="3d")
450     ax3d.plot_surface(x_grid, y_grid, z, cmap="viridis", edgecolor="none",
451     alpha=0.9)
452     ax3d.set_xlabel(x_name)
453     ax3d.set_ylabel(y_name)
454     ax3d.set_zlabel("TTE (h)")
455     ax3d.set_title(f"{pair_name}: response surface")
456     _save_fig(fig, output / f"{pair_name}_surface.png")
457     return df
458
459 def interaction_analysis(output_dir: str | Path, quick: bool = False) -> pd
460 .DataFrame:
461     output = ensure_output_dir(output_dir) / "06_interactions"
462     output.mkdir(parents=True, exist_ok=True)
463     specs = [
464         ("brightness_cpu", "brightness", [0.2, 0.4, 0.6, 0.8, 1.0], "cpu",
465         [0.1, 0.3, 0.5, 0.7, 0.9]),
466         ("cpu_gpu", "cpu", [0.1, 0.3, 0.5, 0.7, 0.9], "gpu", [0.1, 0.3,
467         0.5, 0.7, 0.9]),
468         ("network_signal", "network_traffic", [0.0, 0.25, 0.5, 0.75, 1.0],
469         "signal_quality", [1, 2, 3, 4, 5]),
470         ("temperature_soh", "ambient_temp_c", [-10, 0, 10, 25, 40], "
471         initial_soh", [0.7, 0.8, 0.9, 1.0]),
472     ]
473     frames = []
474     for spec in specs:
475         frame = _interaction_grid(*spec, output=output, quick=quick)
476         frame.insert(0, "interaction", spec[0])
477         frames.append(frame)
478     combined = pd.concat(frames, ignore_index=True, sort=False)
479     save_dataframe(combined, output / "all_interactions.csv")
480     return combined

```

```

475
476 def power_contribution_analysis(output_dir: str | Path) -> pd.DataFrame:
477     output = ensure_output_dir(output_dir) / "07_power_contribution"
478     output.mkdir(parents=True, exist_ok=True)
479     params = default_parameters()
480     rows = []
481     for key, scenario in SCENARIO_LIBRARY.items():
482         load = calc_load_power(behavior_from_scenario(scenario), params.
load)
483         total = load["total"]
484         row = {"scenario_key": key, "scenario": scenario["name"], "
total_power_w": total}
485         for component in ["base", "screen", "cpu", "gpu", "network", "gps",
"background"]:
486             row[f"{component}_power_w"] = load[component]
487             row[f"{component}_share_pct"] = load[component] / total * 100
488         rows.append(row)
489     df = pd.DataFrame(rows)
490     save_dataframe(df, output / "power_contribution.csv")
491
492     components = ["base", "screen", "cpu", "gpu", "network", "gps", "
background"]
493     fig, ax = plt.subplots(figsize=(12.0, 6))
494
495     bottom = np.zeros(len(df))
496     x = np.arange(len(df))
497
498     for component in components:
499         values = df[f"{component}_power_w"].to_numpy()
500         ax.bar(x, values, bottom=bottom, label=component)
501         bottom += values
502
503     ax.set_xticks(x)
504     ax.set_xticklabels(df["scenario"], rotation=35, ha="right")
505
506     ax.set_ylabel("Power (W)")
507     ax.set_title("Component-level power contribution by scenario")
508     ax.grid(axis="y", alpha=0.25)
509
510     ax.legend(ncol=2, bbox_to_anchor=(1.02, 1), loc="upper left", fontsize
=9)
511
512     plt.subplots_adjust(bottom=0.25)
513
514     ax.set_ylim(0, bottom.max() * 1.08)
515
516     _save_fig(fig, output / "power_contribution_stacked.png")
517     return df
518
519
520 def monte_carlo_analysis(
521     output_dir: str | Path,
522     quick: bool = False,
523     n_runs: int | None = None,
524     seed: int = 2026,
525 ) -> pd.DataFrame:
526     output = ensure_output_dir(output_dir) / "08_monte_carlo"
527     output.mkdir(parents=True, exist_ok=True)

```

```

528     n_runs = n_runs or (100 if quick else 500)
529     scenario = get_scenario("game")
530     base_behavior = behavior_from_scenario(scenario)
531     params = default_parameters()
532     params.thermal.T_env = 25.0
533     rng = np.random.default_rng(seed)
534     rows = []
535     for run in range(n_runs):
536         behavior_rng = np.random.default_rng(rng.integers(0, 2**32 - 1))
537         stochastic_behavior = PiecewiseRandomBehavior(base_behavior,
behavior_rng, interval_s=60.0)
538         state = BatteryState(soc=1.0, soh=1.0, temp=25.0)
539         config = _base_config(dt_s=15.0 if quick else 10.0,
record_interval_s=600.0)
540         result = simulate_case(
541             state, stochastic_behavior,
542             params.load, params.electrical, params.thermal,
543             params.soc, params.aging, params.resistance, config,
544         )
545         rows.append({
546             "run": run + 1,
547             "tte_h": result.tte_hours,
548             "stop_reason": result.stop_reason,
549             "final_soc": result.final_state.soc,
550             "final_voltage_v": result.final_state.voltage,
551             "final_temp_c": result.final_state.temp,
552         })
553     df = pd.DataFrame(rows)
554     save_dataframe(df, output / "monte_carlo_runs.csv")
555     summary = pd.DataFrame([
556         "n_runs": n_runs,
557         "mean_tte_h": df["tte_h"].mean(),
558         "std_tte_h": df["tte_h"].std(ddof=1),
559         "ci95_lower_h": df["tte_h"].quantile(0.025),
560         "ci95_upper_h": df["tte_h"].quantile(0.975),
561         "median_tte_h": df["tte_h"].median(),
562     ])
563     save_dataframe(summary, output / "monte_carlo_summary.csv")
564
565     fig, ax = plt.subplots(figsize=(8.0, 5.0))
566     ax.hist(df["tte_h"], bins=20, alpha=0.85)
567     ax.axvline(df["tte_h"].mean(), linestyle="--", label=f"Mean={df['tte_h']
'].mean():.2f} h")
568     ax.set_xlabel("TTE (h)")
569     ax.set_ylabel("Frequency")
570     ax.set_title("Monte Carlo distribution of mobile-game TTE")
571     ax.grid(axis="y", alpha=0.25)
572     ax.legend()
573     _save_fig(fig, output / "monte_carlo_histogram.png")
574
575     fig, ax = plt.subplots(figsize=(5.2, 5.0))
576     ax.boxplot(df["tte_h"], vert=True)
577     ax.set_ylabel("TTE (h)")
578     ax.set_title("Monte Carlo TTE boxplot")
579     ax.grid(axis="y", alpha=0.25)
580     _save_fig(fig, output / "monte_carlo_boxplot.png")
581     return df
582

```

```

583
584 def _load_benchmark_csv(data_path: str | Path) -> pd.DataFrame:
585     raw = pd.read_csv(data_path, header=2)
586     required = {"Scenario", "Initial_SOC", "Time_to_empty_hours"}
587     missing = required - set(raw.columns)
588     if missing:
589         raise ValueError(f"Benchmark CSV missing columns: {sorted(missing)}")
590     return raw.dropna(subset=["Scenario", "Initial_SOC", "Time_to_empty_hours"]).copy()
591
592
593 def validation_analysis(
594     output_dir: str | Path,
595     benchmark_csv: str | Path,
596     quick: bool = False,
597 ) -> pd.DataFrame:
598     output = ensure_output_dir(output_dir) / "09_validation"
599     output.mkdir(parents=True, exist_ok=True)
600     benchmark = _load_benchmark_csv(benchmark_csv)
601     name_map = {scenario["name"]: key for key, scenario in SCENARIO_LIBRARY.items()}
602     rows = []
603     for row in benchmark.itertuples(index=False):
604         if row.Scenario not in name_map:
605             continue
606         key = name_map[row.Scenario]
607         scenario = get_scenario(key)
608         result, _, _ = run_scenario_case(
609             key,
610             initial_soc=float(row.Initial_SOC),
611             initial_soh=float(scenario.get("soh", 1.0)),
612             ambient_temp=float(scenario.get("temperature", 25.0)),
613             config=_base_config(dt_s=15.0 if quick else 10.0),
614         )
615         observed = float(row.Time_to_empty_hours)
616         predicted = result.tte_hours
617         rows.append({
618             "scenario": row.Scenario,
619             "initial_soc": float(row.Initial_SOC),
620             "reference_tte_h": observed,
621             "predicted_tte_h": predicted,
622             "error_h": predicted - observed,
623             "absolute_error_h": abs(predicted - observed),
624             "absolute_percentage_error_pct": abs(predicted - observed) /
625             max(observed, 1e-9) * 100,
626             "stop_reason": result.stop_reason,
627         })
628     df = pd.DataFrame(rows)
629     save_dataframe(df, output / "benchmark_consistency_results.csv")
630     if df.empty:
631         raise ValueError("No benchmark scenarios matched the scenario library")
632     metrics = pd.DataFrame([
633         "MAE_h": df["absolute_error_h"].mean(),
634         "RMSE_h": math.sqrt(np.mean(df["error_h"] ** 2)),
635         "MAPE_pct": df["absolute_percentage_error_pct"].mean(),
636         "n": len(df),

```

```

636         "validation_type": "benchmark/physical-consistency comparison; not
experimental validation",
637     })
638     save_dataframe(metrics, output / "benchmark_consistency_metrics.csv")
639
640     fig, ax = plt.subplots(figsize=(6.0, 6.0))
641     ax.scatter(df["reference_tte_h"], df["predicted_tte_h"], alpha=0.8)
642     limit = max(df["reference_tte_h"].max(), df["predicted_tte_h"].max()) *
1.05
643     ax.plot([0, limit], [0, limit], linestyle="--", label="y = x")
644     ax.set_xlim(0, limit)
645     ax.set_ylim(0, limit)
646     ax.set_xlabel("Reference TTE (h)")
647     ax.set_ylabel("Predicted TTE (h)")
648     ax.set_title("Benchmark consistency of TTE prediction")
649     ax.grid(alpha=0.3)
650     ax.legend()
651     _save_fig(fig, output / "predicted_vs_reference.png")
652     return df
653
654
655 def timestep_analysis(output_dir: str | Path, quick: bool = False) -> pd.
DataFrame:
656     output = ensure_output_dir(output_dir) / "10_timestep"
657     output.mkdir(parents=True, exist_ok=True)
658     dt_values = [0.5, 1.0, 5.0, 10.0, 30.0]
659     if quick:
660         dt_values = [1.0, 5.0, 10.0, 30.0]
661     rows = []
662     for dt in dt_values:
663         start = perf_counter()
664         result, _, _ = run_scenario_case(
665             "game",
666             config=_base_config(dt_s=dt, record_interval_s=max(30.0, dt)),
667         )
668         runtime = perf_counter() - start
669         rows.append({
670             "dt_s": dt,
671             "tte_h": result.tte_hours,
672             "runtime_s": runtime,
673             "stop_reason": result.stop_reason,
674         })
675     df = pd.DataFrame(rows).sort_values("dt_s")
676     reference = float(df.iloc[0]["tte_h"])
677     df["absolute_tte_error_h"] = abs(df["tte_h"] - reference)
678     df["relative_tte_error_pct"] = df["absolute_tte_error_h"] / max(
reference, 1e-9) * 100
679     save_dataframe(df, output / "timestep_stability.csv")
680
681     fig, ax = plt.subplots(figsize=(8.0, 5.0))
682     ax.plot(df["dt_s"], df["relative_tte_error_pct"], marker="o")
683     ax.set_xscale("log")
684     ax.set_xlabel("Time step Δt (s, log scale)")
685     ax.set_ylabel("Relative TTE error (%)")
686     ax.set_title("Numerical time-step stability")
687     ax.grid(alpha=0.3)
688     _save_fig(fig, output / "timestep_error.png")
689

```

```

690     fig, ax = plt.subplots(figsize=(8.0, 5.0))
691     ax.plot(df["dt_s"], df["runtime_s"], marker="o")
692     ax.set_xscale("log")
693     ax.set_xlabel("Time step  $\Delta t$  (s, log scale)")
694     ax.set_ylabel("Computation time (s)")
695     ax.set_title("Accuracy-efficiency trade-off")
696     ax.grid(alpha=0.3)
697     _save_fig(fig, output / "timestep_runtime.png")
698     return df
699
700
701 ANALYSIS_FUNCTIONS = {
702     "soc": initial_soc_analysis,
703     "temperature": temperature_analysis,
704     "soh": soh_analysis,
705     "ablation": model_ablation_analysis,
706     "stop": stop_reason_analysis,
707     "interaction": interaction_analysis,
708     "contribution": power_contribution_analysis,
709     "monte_carlo": monte_carlo_analysis,
710     "timestep": timestep_analysis,
711 }

```

A.1.2 common.py

```

1  """Shared helpers for analysis modules."""
2  from __future__ import annotations
3
4  from copy import deepcopy
5  from dataclasses import dataclass
6  from pathlib import Path
7  from typing import Any
8
9  import pandas as pd
10
11 from BatteryModel.analysis.scenario_analysis import get_scenario,
    behavior_from_scenario
12 from BatteryModel.models.state import BatteryState
13 from BatteryModel.params.load_params import LoadPowerParams
14 from BatteryModel.params.electrical_params import ElectricalParams
15 from BatteryModel.params.thermal_params import ThermalParams
16 from BatteryModel.params.soc_params import SocParams
17 from BatteryModel.params.aging_params import AgingParams
18 from BatteryModel.params.resistance_params import ResistanceParams
19 from BatteryModel.simulation.config import SimulationConfig,
    SimulationResult
20 from BatteryModel.simulation.simulator import simulate_case
21
22
23 @dataclass
24 class ModelParameters:
25     load: LoadPowerParams
26     electrical: ElectricalParams
27     thermal: ThermalParams
28     soc: SocParams
29     aging: AgingParams

```

```

30     resistance: ResistanceParams
31
32
33 def default_parameters() -> ModelParameters:
34     return ModelParameters(
35         load=LoadPowerParams(),
36         electrical=ElectricalParams(),
37         thermal=ThermalParams(),
38         soc=SocParams(),
39         aging=AgingParams(),
40         resistance=ResistanceParams(),
41     )
42
43
44 def ensure_output_dir(path: str | Path) -> Path:
45     output = Path(path)
46     output.mkdir(parents=True, exist_ok=True)
47     return output
48
49
50 def save_dataframe(df: pd.DataFrame, path: Path) -> None:
51     path.parent.mkdir(parents=True, exist_ok=True)
52     df.to_csv(path, index=False, encoding="utf-8-sig")
53
54
55 def run_scenario_case(
56     scenario_name: str | int,
57     *,
58     initial_soc: float = 1.0,
59     initial_soh: float | None = None,
60     ambient_temp: float | None = None,
61     behavior_overrides: dict[str, float] | None = None,
62     config: SimulationConfig | None = None,
63     params: ModelParameters | None = None,
64     behavior_source: Any | None = None,
65 ) -> tuple[SimulationResult, dict, ModelParameters]:
66     scenario = get_scenario(scenario_name)
67     params = deepcopy(params or default_parameters())
68     temp = float(scenario.get("temperature", 25.0) if ambient_temp is None
69 else ambient_temp)
70     soh = float(scenario.get("soh", 1.0) if initial_soh is None else
71 initial_soh)
72     params.thermal.T_env = temp
73
74     state = BatteryState(soc=initial_soc, soh=soh, temp=temp)
75     behavior = behavior_from_scenario(scenario)
76     if behavior_overrides:
77         behavior = behavior.copy_with(**behavior_overrides)
78     behavior = behavior_source or behavior
79
80     config = config or SimulationConfig(dt_s=5.0, record_interval_s=30.0)
81     result = simulate_case(
82         state,
83         behavior,
84         params.load,
85         params.electrical,
86         params.thermal,
87         params.soc,

```



```

86         params.aging,
87         params.resistance,
88         config,
89     )
90     return result, scenario, params

```

A.1.3 scenario_analysis.py

```

1  """Representative phone-use scenarios and helpers."""
2  from __future__ import annotations
3
4  from copy import deepcopy
5  from BatteryModel.models.behavior import UserBehavior
6
7
8  # Values are normalized workloads. ``network_traffic`` uses 0--1 here,
9  # while
10 # the load model remains compatible with the original 0--10 inputs.
11 SCENARIO_LIBRARY = {
12     "office": {
13         "name": "Light daily use",
14         "brightness": 0.35,
15         "cpu": 0.20,
16         "gpu": 0.05,
17         "network_traffic": 0.20,
18         "signal_quality": 5.0,
19         "gps": 0.0,
20         "background": 0.10,
21         "temperature": 25.0,
22         "soh": 1.0,
23     },
24     "video": {
25         "name": "Video streaming (Wi-Fi)",
26         "brightness": 0.65,
27         "cpu": 0.35,
28         "gpu": 0.18,
29         "network_traffic": 0.35,
30         "signal_quality": 5.0,
31         "gps": 0.0,
32         "background": 0.12,
33         "temperature": 25.0,
34         "soh": 1.0,
35     },
36     "navigation": {
37         "name": "Navigation",
38         "brightness": 0.65,
39         "cpu": 0.45,
40         "gpu": 0.10,
41         "network_traffic": 0.45,
42         "signal_quality": 2.0,
43         "gps": 1.0,
44         "background": 0.12,
45         "temperature": 25.0,
46         "soh": 1.0,
47     },
48     "game": {

```

```

48     "name": "Mobile game",
49     "brightness": 0.75,
50     "cpu": 0.85,
51     "gpu": 0.75,
52     "network_traffic": 0.15,
53     "signal_quality": 4.0,
54     "gps": 0.0,
55     "background": 0.15,
56     "temperature": 25.0,
57     "soh": 1.0,
58 },
59 "standby": {
60     "name": "Standby",
61     "brightness": 0.0,
62     "cpu": 0.05,
63     "gpu": 0.0,
64     "network_traffic": 0.01,
65     "signal_quality": 5.0,
66     "gps": 0.0,
67     "background": 0.08,
68     "temperature": 25.0,
69     "soh": 1.0,
70 },
71 "weak_signal": {
72     "name": "Weak-signal social/video",
73     "brightness": 0.45,
74     "cpu": 0.30,
75     "gpu": 0.10,
76     "network_traffic": 0.35,
77     "signal_quality": 1.0,
78     "gps": 0.0,
79     "background": 0.15,
80     "temperature": 25.0,
81     "soh": 1.0,
82 },
83 "cold_aged": {
84     "name": "Cold aged light use",
85     "brightness": 0.35,
86     "cpu": 0.20,
87     "gpu": 0.05,
88     "network_traffic": 0.20,
89     "signal_quality": 5.0,
90     "gps": 0.0,
91     "background": 0.10,
92     "temperature": 10.0,
93     "soh": 0.83,
94 },
95 }
96
97 # Legacy integer choices used by the original interactive main.py.
98 SCENARIOS = {
99     1: deepcopy(SCENARIO_LIBRARY["office"]),
100     2: deepcopy(SCENARIO_LIBRARY["video"]),
101     3: deepcopy(SCENARIO_LIBRARY["navigation"]),
102     4: deepcopy(SCENARIO_LIBRARY["game"]),
103     5: deepcopy(SCENARIO_LIBRARY["standby"]),
104     6: deepcopy(SCENARIO_LIBRARY["weak_signal"]),
105     7: deepcopy(SCENARIO_LIBRARY["cold_aged"]),

```

```

106 }
107
108
109 def get_scenario(name_or_id: str | int) -> dict:
110     if isinstance(name_or_id, int) or str(name_or_id).isdigit():
111         key = int(name_or_id)
112         if key not in SCENARIOS:
113             raise KeyError(f"Unknown scenario id: {key}")
114         return deepcopy(SCENARIOS[key])
115     key = str(name_or_id).strip().lower()
116     if key not in SCENARIO_LIBRARY:
117         valid = ", ".join(SCENARIO_LIBRARY)
118         raise KeyError(f"Unknown scenario '{name_or_id}'. Valid names: {valid}")
119     return deepcopy(SCENARIO_LIBRARY[key])
120
121
122 def behavior_from_scenario(scenario: dict) -> UserBehavior:
123     return UserBehavior(
124         brightness=scenario["brightness"],
125         cpu=scenario["cpu"],
126         gpu=scenario["gpu"],
127         network_traffic=scenario["network_traffic"],
128         signal_quality=scenario["signal_quality"],
129         gps=scenario["gps"],
130         background=scenario["background"],
131     )

```

A.1.4 sensitivity.py

```

1 """Single-factor sensitivity analysis using actual simulated TTE."""
2 from __future__ import annotations
3
4 from pathlib import Path
5 import matplotlib
6 matplotlib.use("Agg")
7 import matplotlib.pyplot as plt
8 import pandas as pd
9
10 from BatteryModel.analysis.common import ensure_output_dir,
11     run_scenario_case, save_dataframe
12 from BatteryModel.simulation.config import SimulationConfig
13
14 DEFAULT_VALUES = {
15     "brightness": [0.2, 0.4, 0.6, 0.8, 1.0],
16     "cpu": [0.1, 0.3, 0.5, 0.7, 0.9],
17     "gpu": [0.0, 0.2, 0.4, 0.6, 0.8],
18     "network_traffic": [0.0, 0.25, 0.5, 0.75, 1.0],
19     "signal_quality": [1, 2, 3, 4, 5],
20     "gps": [0.0, 0.25, 0.5, 0.75, 1.0],
21     "background": [0.0, 0.1, 0.2, 0.3, 0.4],
22 }
23
24
25 def run_sensitivity(

```

```

26     param_name: str,
27     values: list[float] | None = None,
28     *,
29     scenario: str = "game",
30     q_nom: float = 3.45,
31     dt: float = 5.0,
32     total_time: float = 24 * 3600,
33     **_legacy_params,
34 ) -> list[dict]:
35     if param_name not in DEFAULT_VALUES:
36         raise ValueError(f"Unsupported parameter: {param_name}")
37     values = list(values or DEFAULT_VALUES[param_name])
38     rows = []
39     for value in values:
40         config = SimulationConfig(
41             q_nom_ah=q_nom,
42             dt_s=dt,
43             max_time_s=total_time,
44             record_interval_s=max(30.0, dt),
45         )
46         result, scenario_cfg, _ = run_scenario_case(
47             scenario,
48             behavior_overrides={param_name: value},
49             config=config,
50         )
51         rows.append({
52             "parameter": param_name,
53             "value": value,
54             "scenario": scenario_cfg["name"],
55             "tte_h": result.tte_hours,
56             "stop_reason": result.stop_reason,
57             "final_soc": result.final_state.soc,
58             "final_soh": result.final_state.soh,
59             "final_temp_c": result.final_state.temp,
60             "final_voltage_v": result.final_state.voltage,
61         })
62     return rows
63
64
65 def plot_sensitivity(
66     results: list[dict],
67     param_name: str,
68     output_dir: str | Path = "results/sensitivity",
69 ) -> pd.DataFrame:
70     output = ensure_output_dir(output_dir)
71     df = pd.DataFrame(results).sort_values("value")
72     save_dataframe(df, output / f"{param_name}_sensitivity.csv")
73
74     plot_specs = [
75         ("tte_h", "Actual TTE (h)", f"{param_name}_tte.png"),
76         ("final_temp_c", "Final temperature (°C)", f"{param_name}"
77         _temperature.png"),
78         ("final_soc", "Final SOC", f"{param_name}_soc.png"),
79         ("final_voltage_v", "Final terminal voltage (V)", f"{param_name}"
80         _voltage.png"),
81     ]
82     for column, ylabel, filename in plot_specs:
83         fig, ax = plt.subplots(figsize=(8.0, 5.0))

```

```

82     ax.plot(df["value"], df[column], marker="o")
83     ax.set_xlabel(param_name)
84     ax.set_ylabel(ylabel)
85     ax.set_title(f"Single-factor sensitivity: {param_name}")
86     ax.grid(alpha=0.3)
87     fig.tight_layout()
88     fig.savefig(output / filename, dpi=300, bbox_inches="tight")
89     plt.close(fig)
90     return df

```

A.2 Model Layer

A.2.1 behavior.py

```

1  """User workload definitions used by the battery simulation."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass, replace
5  from typing import Dict
6  import numpy as np
7
8
9  @dataclass
10 class UserBehavior:
11     """Normalized phone workload.
12
13     Parameters are expected in [0, 1], except ``network_traffic`` which
14     also
15     accepts the legacy 0--10 scale, and ``signal_quality`` which uses 1--5
16     (1 = very weak, 5 = excellent).
17     """
18
19     brightness: float = 0.6
20     cpu: float = 0.35
21     gpu: float = 0.10
22     network_traffic: float = 0.5
23     signal_quality: float = 4.0
24     gps: float = 0.0
25     background: float = 0.08
26
27     def clipped(self) -> "UserBehavior":
28         traffic = float(self.network_traffic)
29         if traffic > 1.0: # backward compatible with the old 0--10 inputs
30             traffic /= 10.0
31         return UserBehavior(
32             brightness=float(np.clip(self.brightness, 0.0, 1.0)),
33             cpu=float(np.clip(self.cpu, 0.0, 1.0)),
34             gpu=float(np.clip(self.gpu, 0.0, 1.0)),
35             network_traffic=float(np.clip(traffic, 0.0, 1.0)),
36             signal_quality=float(np.clip(self.signal_quality, 1.0, 5.0)),
37             gps=float(np.clip(self.gps, 0.0, 1.0)),
38             background=float(np.clip(self.background, 0.0, 1.0)),
39         )
40
41     def copy_with(self, **changes: float) -> "UserBehavior":

```

```

41         return replace(self, **changes)
42
43
44 class PiecewiseRandomBehavior:
45     """Piecewise-constant stochastic workload for Monte Carlo analysis.
46
47     A new sample is generated every ``interval_s`` seconds. Samples follow
48     clipped normal distributions around the base behavior, which captures
49     both
50     run-to-run variability and temporal fluctuations without making the
51     simulation unnecessarily expensive.
52     """
53
54     def __init__(
55         self,
56         base: UserBehavior,
57         rng: np.random.Generator,
58         interval_s: float = 60.0,
59         relative_sigma: Dict[str, float] | None = None,
60     ) -> None:
61         self.base = base.clipped()
62         self.rng = rng
63         self.interval_s = max(float(interval_s), 1.0)
64         self.relative_sigma = relative_sigma or {
65             "brightness": 0.08,
66             "cpu": 0.15,
67             "gpu": 0.18,
68             "network_traffic": 0.20,
69             "signal_quality": 0.10,
70             "background": 0.15,
71         }
72         self._bin = None
73         self._current = self.base
74
75     def __call__(self, time_s: float) -> UserBehavior:
76         current_bin = int(time_s // self.interval_s)
77         if current_bin != self._bin:
78             self._bin = current_bin
79             values = {}
80             for name in (
81                 "brightness", "cpu", "gpu", "network_traffic",
82                 "signal_quality", "background",
83             ):
84                 mean = float(getattr(self.base, name))
85                 sigma = self.relative_sigma.get(name, 0.0) * max(abs(mean),
86                     0.05)
87                 values[name] = float(self.rng.normal(mean, sigma))
88                 values["gps"] = self.base.gps
89             self._current = self.base.copy_with(**values).clipped()
90         return self._current

```

A.2.2 state.py

```

1 """Battery state container."""
2 from __future__ import annotations
3

```

```

4 from dataclasses import dataclass
5
6
7 @dataclass
8 class BatteryState:
9     time: float = 0.0
10    soc: float = 1.0
11    soh: float = 1.0
12    temp: float = 25.0
13    voltage: float = 4.2
14    current: float = 0.0
15    v1: float = 0.0
16    v2: float = 0.0
17
18    def copy(self) -> "BatteryState":
19        return BatteryState(**self.__dict__)

```

A.3 Parameter Layer

A.3.1 aging_params.py

```

1 class AgingParams:
2
3     def __init__(self):
4
5         # 老化速率系数
6         self.k0 = 1e-8
7
8         # 活化能 ( $J/mol$ )
9         self.Ea = 35000.0
10
11        # 气体常数
12        self.Rg = 8.314
13
14        # C倍率指数
15        self.mu = 1.2
16
17        # SOC偏离惩罚
18        self.rho = 2.0
19
20        # 最佳SOC区间
21        self.z_mid = 0.5

```

A.3.2 electrical_params.py

```

1 class ElectricalParams:
2
3     def __init__(self):
4
5         self.R0_ref = 0.015
6
7         self.R1_ref = 0.005
8         self.C1 = 1000

```

```

9
10     self.R2_ref = 0.010
11     self.C2 = 5000

```

A.3.3 load_params.py

```

1  """Parameters of the component-level phone power model."""
2
3
4  class LoadPowerParams:
5      def __init__(self, p0: float = 0.35):
6          # Base/platform power (W)
7          self.p0 = p0
8
9          # Display
10         self.k0 = 1.2
11         self.As = 1.0
12         self.gamma = 1.4
13
14         # CPU/GPU dynamic power
15         self.kc = 1.5
16         self.alpha = 2.0
17         self.kg = 1.8
18         self.beta = 2.2
19
20         # Radio: traffic is normalized to 0--1; weak-signal penalty is
21         0--1.
22         self.p_net0 = 0.25
23         self.kr = 0.65
24         self.ks = 0.65
25         self.s_ref = 0.7 # retained for compatibility; no longer used
26         directly
27
28         self.k_gps = 0.35
29         self.k_bg = 0.30

```

A.3.4 resistance_params.py

```

1  class ResistanceParams:
2
3      def __init__(self):
4
5          self.T_ref = 25.0
6
7          self.z_ref = 0.5
8
9          # SOH影响系数
10         self.a = 3.0
11
12         # 温度影响系数
13         self.lam = 0.03
14
15         # SOC影响系数
16         self.c = 1.0

```


A.3.5 soc_params.py

```
1 class SocParams:
2
3     def __init__(self):
4
5         # 库仑效率
6         self.eta = 1.0
```

A.3.6 thermal_params.py

```
1 """Thermal parameters for the coupled battery/device model."""
2
3
4 class ThermalParams:
5     def __init__(self):
6         # Lumped battery/device thermal capacity (J/K)
7         self.C_th = 500.0
8
9         # Effective heat-transfer coefficient to ambient (W/K). The
10        original
11        # value 5 W/K suppressed almost all temperature dynamics; 0.15 W/K
12        # corresponds to a thermal time constant of roughly 0.9 h.
13        self.h = 0.15
14
15        # Fraction of phone load power thermally coupled into the battery
16        pack.
17        # This complements I^2R heat and captures CPU/GPU/display heat
18        transfer.
19        self.device_heat_fraction = 0.18
20
21        self.T_env = 25.0
22
23        # Low-temperature usable-capacity correction
24        self.T_ref = 25.0
25        self.a_low = 0.01
26
27        # Reserved thermal-throttling parameters
28        self.T_hot = 45.0
29        self.a_hot = 0.02
30
31        # Thermal protection threshold
32        self.T_max = 60.0
```

A.4 Physics Layer

A.4.1 electrical.py

```
1 """Thevenin 2RC electrical model."""
2 from __future__ import annotations
3
4 from math import sqrt, exp
5 from BatteryModel.models.state import BatteryState
```

```

6
7
8 def ocv(soc: float) -> float:
9     """Smooth monotonic OCV approximation for a single Li-ion cell."""
10    z = min(max(float(soc), 0.0), 1.0)
11    # A mild S-shape is more realistic than the original strictly linear
OCV.
12    return 3.0 + 1.05 * z + 0.15 * z * z
13
14
15 def calc_current(state: BatteryState, p_load: float, r0: float) -> float:
16    """Solve  $P = I(U_{oc} - V_1 - V_2 - I R_0)$  for the physical low-current root
."""
17    if p_load <= 0:
18        return 0.0
19    a = max(ocv(state.soc) - state.v1 - state.v2, 1e-6)
20    r0 = max(float(r0), 1e-9)
21    discriminant = a * a - 4.0 * r0 * p_load
22    if discriminant <= 0.0:
23        # Requested power exceeds the static maximum-power point. Returning
the
24        # MPP current makes the terminal voltage collapse and triggers
cutoff.
25        return a / (2.0 * r0)
26    return (a - sqrt(discriminant)) / (2.0 * r0)
27
28
29 def _update_rc_voltage(previous: float, current: float, resistance: float,
30 capacitance: float, dt: float) -> float:
31    """Exact zero-order-hold discretization of a first-order RC branch.
32
33    This avoids the explicit-Euler instability that occurred when dt
exceeded
34    the fast branch time constant  $R_1 C_1$  (5 s in the default parameters).
"""
35    tau = max(resistance * capacitance, 1e-12)
36    decay = exp(-dt / tau)
37    return previous * decay + current * resistance * (1.0 - decay)
38
39
40 def update_v1(state: BatteryState, r1: float, c1: float, dt: float) -> None:
41    :
42    state.v1 = _update_rc_voltage(state.v1, state.current, r1, c1, dt)
43
44 def update_v2(state: BatteryState, r2: float, c2: float, dt: float) -> None:
45    :
46    state.v2 = _update_rc_voltage(state.v2, state.current, r2, c2, dt)
47
48 def update_voltage(state: BatteryState, r0: float) -> None:
49    state.voltage = ocv(state.soc) - state.current * r0 - state.v1 - state.
    v2

```

A.4.2 load.py

```

1  """Phone component power model."""
2  from __future__ import annotations
3
4  from BatteryModel.models.behavior import UserBehavior
5  from BatteryModel.params.load_params import LoadPowerParams
6
7
8  def normalize_traffic(value: float) -> float:
9      """Accept both normalized 0--1 and legacy 0--10 traffic inputs."""
10     value = float(value)
11     if value > 1.0:
12         value /= 10.0
13     return min(max(value, 0.0), 1.0)
14
15
16 def weak_signal_penalty(signal_quality: float, _legacy_s_ref: float = 0.7)
-> float:
17     """Return a 0--1 radio penalty for signal quality on the 1--5 scale."""
18     quality = min(max(float(signal_quality), 1.0), 5.0)
19     return (5.0 - quality) / 4.0
20
21
22 def calc_load_power(user: UserBehavior, params: LoadPowerParams) -> dict[
str, float]:
23     user = user.clipped()
24     traffic = normalize_traffic(user.network_traffic)
25
26     p_screen = params.k0 * params.As * user.brightness ** params.gamma
27     p_cpu = params.kc * user.cpu ** params.alpha
28     p_gpu = params.kg * user.gpu ** params.beta
29
30     penalty = weak_signal_penalty(user.signal_quality, params.s_ref)
31     p_net = params.p_net0 + params.kr * traffic + params.ks * penalty
32     p_gps = params.k_gps * user.gps
33     p_bg = params.k_bg * user.background
34
35     p_load = params.p0 + p_screen + p_cpu + p_gpu + p_net + p_gps + p_bg
36
37     return {
38         "total": float(p_load),
39         "base": float(params.p0),
40         "screen": float(p_screen),
41         "cpu": float(p_cpu),
42         "gpu": float(p_gpu),
43         "network": float(p_net),
44         "gps": float(p_gps),
45         "background": float(p_bg),
46     }

```

A.4.3 resistance.py

```

1  from math import exp
2
3  from BatteryModel.models.state import BatteryState
4  from BatteryModel.params.resistance_params import ResistanceParams
5

```

```

6 def calc_resistance(
7     r_ref: float,
8     state: BatteryState,
9     params: ResistanceParams
10 ) -> float:
11
12     return (
13         r_ref
14         * (
15             1
16             + params.a
17             * (1 - state.soh)
18         )
19         * exp(
20             params.lam
21             * (
22                 params.T_ref
23                 - state.temp
24             )
25         )
26         * (
27             1
28             + params.c
29             * (
30                 state.soc
31                 - params.z_ref
32             ) ** 2
33         )
34     )

```

A.4.4 soc.py

```

1 """Coulomb-counting SOC dynamics with optional temperature correction."""
2 from __future__ import annotations
3
4 from BatteryModel.models.state import BatteryState
5 from BatteryModel.params.soc_params import SocParams
6 from BatteryModel.params.thermal_params import ThermalParams
7 from BatteryModel.physics.thermal import effective_capacity
8
9
10 def soc_derivative(
11     state: BatteryState,
12     q_nom: float,
13     thermal_params: ThermalParams,
14     soc_params: SocParams,
15     use_temperature_effect: bool = True,
16 ) -> float:
17     if use_temperature_effect:
18         q_eff = effective_capacity(q_nom, state, thermal_params)
19     else:
20         q_eff = q_nom * state.soh
21
22     if q_eff <= 0:
23         return 0.0
24

```

```

25     return -soc_params.eta * state.current / (3600.0 * q_eff)
26
27
28 def update_soc(
29     state: BatteryState,
30     q_nom: float,
31     thermal_params: ThermalParams,
32     soc_params: SocParams,
33     dt: float,
34     use_temperature_effect: bool = True,
35 ) -> None:
36     state.soc += soc_derivative(
37         state,
38         q_nom,
39         thermal_params,
40         soc_params,
41         use_temperature_effect=use_temperature_effect,
42     ) * dt
43     state.soc = max(0.0, min(1.0, state.soc))

```

A.4.5 soh.py

```

1 from math import exp
2
3 from BatteryModel.models.state import BatteryState
4 from BatteryModel.params.aging_params import AgingParams
5
6 def soc_stress_factor(
7     soc: float,
8     params: AgingParams
9 ) -> float:
10
11     return (
12         1
13         + params.rho
14         * (soc - params.z_mid) ** 2
15     )
16
17 def soh_derivative(
18     state: BatteryState,
19     q_nom: float,
20     params: AgingParams
21 ) -> float:
22
23     arrhenius = exp(
24         -params.Ea
25         / (
26             params.Rg
27             * (
28                 state.temp
29                 + 273.15
30             )
31         )
32     )
33
34     c_rate = (

```

```

35         abs(state.current)
36         / q_nom
37     ) ** params.mu
38
39     soc_factor = soc_stress_factor(
40         state.soc,
41         params
42     )
43
44     dsoh_dt = (
45         -params.k0
46         / q_nom
47         * arrhenius
48         * c_rate
49         * soc_factor
50     )
51
52     return dsoh_dt
53
54 def update_soh(
55     state: BatteryState,
56     q_nom: float,
57     params: AgingParams,
58     dt: float
59 ):
60
61     dsoh_dt = soh_derivative(
62         state,
63         q_nom,
64         params
65     )
66
67     state.soh += dsoh_dt * dt
68
69     state.soh = max(
70         0.0,
71         min(
72             1.0,
73             state.soh
74         )
75     )

```

A.4.6 thermal.py

```

1  """Lumped electro-thermal battery model."""
2  from __future__ import annotations
3
4  from BatteryModel.models.state import BatteryState
5  from BatteryModel.params.thermal_params import ThermalParams
6
7
8  def heat_generation(
9      state: BatteryState,
10     r0: float,
11     r1: float,
12     r2: float,

```

```

13     p_load: float = 0.0,
14     device_heat_fraction: float = 0.0,
15 ) -> float:
16     electrical_heat = (
17         state.current ** 2 * r0
18         + state.v1 ** 2 / max(r1, 1e-12)
19         + state.v2 ** 2 / max(r2, 1e-12)
20     )
21     coupled_device_heat = max(device_heat_fraction, 0.0) * max(p_load, 0.0)
22     return electrical_heat + coupled_device_heat
23
24
25 def update_temperature(
26     state: BatteryState,
27     params: ThermalParams,
28     r0: float,
29     r1: float,
30     r2: float,
31     dt: float,
32     p_load: float = 0.0,
33 ) -> None:
34     q_gen = heat_generation(
35         state,
36         r0,
37         r1,
38         r2,
39         p_load=p_load,
40         device_heat_fraction=params.device_heat_fraction,
41     )
42     dT_dt = (q_gen - params.h * (state.temp - params.T_env)) / params.C_th
43     state.temp += dT_dt * dt
44
45
46 def low_temp_factor(temp: float, params: ThermalParams) -> float:
47     return max(0.05, 1.0 - params.a_low * max(0.0, params.T_ref - temp))
48
49
50 def hot_temp_factor(temp: float, params: ThermalParams) -> float:
51     return max(0.0, 1.0 - params.a_hot * max(0.0, temp - params.T_hot))
52
53
54 def effective_capacity(q_nom: float, state: BatteryState, params:
55     ThermalParams) -> float:
56     return q_nom * state.soh * low_temp_factor(state.temp, params)

```

A.4.7 tte.py

```

1 from BatteryModel.models.state import BatteryState
2
3
4 def calc_tte(
5     state: BatteryState,
6     q_nom: float,
7     power: float
8 ):
9     """

```

```

10     返回剩余时间 (秒)
11     """
12
13     if power <= 0:
14         return float("inf")
15
16     # 剩余电量 (Wh)
17
18     remaining_energy = (
19         state.soc
20         * state.soh
21         * q_nom
22         * state.voltage
23     )
24
25     # TTE (h)
26
27     tte_hour = (
28         remaining_energy
29         / power
30     )
31
32     return tte_hour * 3600

```

A.5 Simulation Layer

A.5.1 simulator.py

```

1  """Core battery digital-twin simulation."""
2  from __future__ import annotations
3
4  from collections.abc import Callable
5  from typing import Union
6
7  from BatteryModel.models.state import BatteryState
8  from BatteryModel.models.behavior import UserBehavior
9  from BatteryModel.physics.load import calc_load_power
10 from BatteryModel.physics.electrical import calc_current, update_v1,
    update_v2, update_voltage, ocv
11 from BatteryModel.physics.tte import calc_tte
12 from BatteryModel.physics.thermal import update_temperature
13 from BatteryModel.physics.soc import update_soc
14 from BatteryModel.physics.soh import update_soh
15 from BatteryModel.physics.resistance import calc_resistance
16 from BatteryModel.simulation.config import SimulationConfig,
    SimulationResult
17
18 BehaviorSource = Union[UserBehavior, Callable[[float], UserBehavior]]
19
20
21 def _resolve_behavior(source: BehaviorSource, time_s: float) ->
    UserBehavior:
22     behavior = source(time_s) if callable(source) else source
23     if not isinstance(behavior, UserBehavior):
24         raise TypeError("behavior source must return UserBehavior")

```



```

25     return behavior.clipped()
26
27
28 def _stop_reason(state: BatteryState, config: SimulationConfig, t_max:
float) -> str | None:
29     if config.stop_on_soc and state.soc <= config.soc_min:
30         return "SOC threshold"
31     if config.stop_on_voltage and state.voltage <= config.voltage_cutoff_v:
32         return "voltage cutoff"
33     if config.stop_on_temperature and state.temp >= t_max:
34         return "thermal protection"
35     return None
36
37
38 def simulate_case(
39     state: BatteryState,
40     behavior: BehaviorSource,
41     load_params,
42     electrical_params,
43     thermal_params,
44     soc_params,
45     aging_params,
46     resistance_params,
47     config: SimulationConfig | None = None,
48 ) -> SimulationResult:
49     """Run a simulation until a physical stop condition or ``max_time_s``.
50
51     The original project estimated TTE from instantaneous energy/power
52     after a
53     fixed one-hour run. This function instead reports the actual simulated
54     time
55     required to reach SOC, voltage, or thermal limits.
56     """
57     config = config or SimulationConfig()
58     config.validate()
59
60     # Keep initial voltage consistent with the selected initial SOC.
61     state.soc = min(max(float(state.soc), 0.0), 1.0)
62     state.soh = min(max(float(state.soh), 0.0), 1.0)
63     state.voltage = ocv(state.soc) - state.v1 - state.v2
64
65     history: list[dict] = []
66     record_every = max(1, int(round(config.record_interval_s / config.dt_s)
67 ))
68     max_steps = int(config.max_time_s / config.dt_s)
69     stop_reason = _stop_reason(state, config, thermal_params.T_max)
70
71     for step in range(max_steps):
72         if stop_reason is not None:
73             break
74
75         user = _resolve_behavior(behavior, state.time)
76         load_info = calc_load_power(user, load_params)
77         p_load = load_info["total"]
78
79         if config.use_dynamic_resistance:
80             r0 = calc_resistance(electrical_params.R0_ref, state,
81 resistance_params)

```

```

78         r1 = calc_resistance(electrical_params.R1_ref, state,
resistance_params)
79         r2 = calc_resistance(electrical_params.R2_ref, state,
resistance_params)
80     else:
81         r0 = electrical_params.R0_ref
82         r1 = electrical_params.R1_ref
83         r2 = electrical_params.R2_ref
84
85     state.current = calc_current(state, p_load, max(r0, 1e-9))
86
87     if config.use_2rc:
88         update_v1(state, max(r1, 1e-9), electrical_params.C1, config.
dt_s)
89         update_v2(state, max(r2, 1e-9), electrical_params.C2, config.
dt_s)
90     else:
91         state.v1 = 0.0
92         state.v2 = 0.0
93
94     update_voltage(state, r0)
95
96     if config.use_thermal:
97         update_temperature(state, thermal_params, r0, r1, r2, config.
dt_s, p_load=p_load)
98     else:
99         state.temp = thermal_params.T_env
100
101     # SOH always affects usable capacity when supplied as an initial
state;
102     # ``use_soh`` controls only dynamic degradation during this
discharge.
103     update_soc(
104         state,
105         config.q_nom_ah,
106         thermal_params,
107         soc_params,
108         config.dt_s,
109         use_temperature_effect=config.use_thermal,
110     )
111     if config.use_soh:
112         update_soh(state, config.q_nom_ah, aging_params, config.dt_s)
113
114     state.time += config.dt_s
115     stop_reason = _stop_reason(state, config, thermal_params.T_max)
116
117     if step % record_every == 0 or stop_reason is not None or step ==
max_steps - 1:
118         row = {
119             "time": state.time,
120             "soc": state.soc,
121             "soh": state.soh,
122             "temp": state.temp,
123             "voltage": state.voltage,
124             "current": state.current,
125             "v1": state.v1,
126             "v2": state.v2,
127             "R0": r0,

```

```

128         "R1": r1,
129         "R2": r2,
130         "power": p_load,
131         "tte_estimate": calc_tte(state, config.q_nom_ah, p_load),
132         "brightness": user.brightness,
133         "cpu_load": user.cpu,
134         "gpu_load": user.gpu,
135         "network_traffic": user.network_traffic,
136         "signal_quality": user.signal_quality,
137         "gps_flag": user.gps,
138     }
139     for key, value in load_info.items():
140         if key != "total":
141             row[f"power_{key}"] = value
142     history.append(row)
143
144     if stop_reason is None:
145         stop_reason = "maximum simulation time"
146
147     if history:
148         history[-1]["stop_reason"] = stop_reason
149         history[-1]["actual_tte_s"] = state.time
150
151     return SimulationResult(
152         history=history,
153         stop_reason=stop_reason,
154         elapsed_time_s=state.time,
155         final_state=state.copy(),
156     )
157
158
159 def simulate(
160     state: BatteryState,
161     user: BehaviorSource,
162     load_params,
163     electrical_params,
164     thermal_params,
165     soc_params,
166     aging_params,
167     resistance_params,
168     q_nom: float = 3.45,
169     dt: float = 1.0,
170     total_time: float = 3600.0,
171 ):
172     """Backward-compatible wrapper returning only the history list."""
173     config = SimulationConfig(
174         q_nom_ah=q_nom,
175         dt_s=dt,
176         max_time_s=total_time,
177         record_interval_s=dt,
178     )
179     result = simulate_case(
180         state, user, load_params, electrical_params, thermal_params,
181         soc_params, aging_params, resistance_params, config,
182     )
183     # Preserve the old key used by existing plotting code.
184     for row in result.history:
185         row["tte"] = row["tte_estimate"]

```

186 `return result.history`

A.5.2 config.py

```
1 """Simulation configuration and result types."""
2 from __future__ import annotations
3
4 from dataclasses import dataclass
5 from typing import Any
6
7
8 @dataclass
9 class SimulationConfig:
10     q_nom_ah: float = 3.45
11     dt_s: float = 1.0
12     max_time_s: float = 24 * 3600.0
13     record_interval_s: float = 10.0
14
15     soc_min: float = 0.03
16     voltage_cutoff_v: float = 3.0
17
18     use_2rc: bool = True
19     use_thermal: bool = True
20     use_soh: bool = True
21     use_dynamic_resistance: bool = True
22
23     stop_on_soc: bool = True
24     stop_on_voltage: bool = True
25     stop_on_temperature: bool = True
26
27     def validate(self) -> None:
28         if self.q_nom_ah <= 0:
29             raise ValueError("q_nom_ah must be positive")
30         if self.dt_s <= 0 or self.max_time_s <= 0:
31             raise ValueError("dt_s and max_time_s must be positive")
32         if not 0 <= self.soc_min < 1:
33             raise ValueError("soc_min must be in [0, 1)")
34
35
36 @dataclass
37 class SimulationResult:
38     history: list[dict[str, Any]]
39     stop_reason: str
40     elapsed_time_s: float
41     final_state: Any
42
43     @property
44     def tte_hours(self) -> float:
45         return self.elapsed_time_s / 3600.0
```

A.6 Main Entry

A.6.1 generate_all_figures_1.py

```

1  """One-click reproduction of every figure used in the Word analysis report.
2
3  Run from any directory:
4      python generate_all_figures.py
5
6  The default ``report`` profile reproduces the figures delivered with the
7      report:
8  - all deterministic analyses use ``--quick``;
9  - Monte Carlo uses 30 runs with the fixed seed embedded in the model;
10 - outputs are written into ``generated_figures`` beside this script.
11 """
12
13 from __future__ import annotations
14
15 import argparse
16 import csv
17 import subprocess
18 import sys
19 from pathlib import Path
20 from time import perf_counter
21
22 PROJECT_ROOT = Path(__file__).resolve().parent
23 PACKAGE_DIR = PROJECT_ROOT / "BatteryModel"
24
25 EXPECTED_PNG = [
26     "01_initial_soc/initial_soc_tte.png",
27     "01_initial_soc/initial_soc_scenario_comparison.png",
28     "02_temperature/temperature_tte.png",
29     "02_temperature/temperature_scenario_heatmap.png",
30     "02_temperature/game_voltage_by_temperature.png",
31     "02_temperature/game_temperature_trajectories.png",
32     "02_temperature/game_soc_by_temperature.png",
33     "02_temperature/temperature_relative_aging_rate.png",
34     "03_soh/soh_tte.png",
35     "03_soh/soh_voltage_trajectories.png",
36     "04_ablation/model_ablation.png",
37     "05_stop_reason/stop_reason_map.png",
38     "06_interactions/brightness_cpu_heatmap.png",
39     "06_interactions/brightness_cpu_contour.png",
40     "06_interactions/brightness_cpu_surface.png",
41     "06_interactions/cpu_gpu_heatmap.png",
42     "06_interactions/cpu_gpu_contour.png",
43     "06_interactions/cpu_gpu_surface.png",
44     "06_interactions/network_signal_heatmap.png",
45     "06_interactions/network_signal_contour.png",
46     "06_interactions/network_signal_surface.png",
47     "06_interactions/temperature_soh_heatmap.png",
48     "06_interactions/temperature_soh_contour.png",
49     "06_interactions/temperature_soh_surface.png",
50     "07_power_contribution/power_contribution_stacked.png",
51     "08_monte_carlo/monte_carlo_histogram.png",
52     "08_monte_carlo/monte_carlo_boxplot.png",
53     "09_validation/predicted_vs_reference.png",
54     "10_timestep/timestep_error.png",
55     "10_timestep/timestep_runtime.png",
56 ]
57
58 STEPS = [

```

```

57     ("soc", "初始 SOC—TTE 分析"),
58     ("temperature", "环境温度分析"),
59     ("soh", "SOH 老化分析"),
60     ("ablation", "模型增量与消融分析"),
61     ("stop", "TTE 终止原因分析"),
62     ("interaction", "双因素交互分析"),
63     ("contribution", "场景功耗贡献分解"),
64     ("monte_carlo", "蒙特卡洛不确定性分析"),
65     ("validation", "基准一致性分析"),
66     ("timestep", "时间步长稳定性分析"),
67 ]
68
69
70 def build_parser() -> argparse.ArgumentParser:
71     parser = argparse.ArgumentParser(description="生成报告中使用的全部电池 TTE
72     分析图")
73     parser.add_argument(
74         "--profile",
75         choices=["report", "full"],
76         default="report",
77         help="report=复现已交付报告图; full=更细时间步和默认500次蒙特卡洛",
78     )
79     parser.add_argument(
80         "--output",
81         default=str(PROJECT_ROOT / "generated_figures"),
82         help="结果输出目录",
83     )
84     parser.add_argument(
85         "--mc-runs",
86         type=int,
87         default=None,
88         help="覆盖蒙特卡洛次数; report 默认30, full 默认500",
89     )
90     parser.add_argument(
91         "--only",
92         choices=[name for name, _ in STEPS],
93         default=None,
94         help="只运行某一个分析模块",
95     )
96     return parser
97
98 def run_step(name: str, label: str, output: Path, profile: str, mc_runs:
99 int) -> float:
100     command = [
101         sys.executable,
102         "-m",
103         "BatteryModel.main",
104         "--analysis",
105         name,
106         "--output",
107         str(output),
108     ]
109     if profile == "report":
110         command.append("--quick")
111     if name == "monte_carlo":
112         command.extend(["--mc-runs", str(mc_runs)])

```

```

113     print(f"\n[{label}] 开始", flush=True)
114     print("命令: " + " ".join(command), flush=True)
115     started = perf_counter()
116     log_dir = output / "_run_logs"
117     log_dir.mkdir(parents=True, exist_ok=True)
118     log_path = log_dir / f"{name}.log"
119     try:
120         with log_path.open("w", encoding="utf-8") as log_handle:
121             subprocess.run(
122                 command,
123                 cwd=PROJECT_ROOT,
124                 check=True,
125                 stdout=log_handle,
126                 stderr=subprocess.STDOUT,
127                 timeout=1800,
128             )
129     except (subprocess.CalledProcessError, subprocess.TimeoutExpired) as exc:
130         tail = ""
131         if log_path.is_file():
132             lines = log_path.read_text(encoding="utf-8", errors="replace").
splitlines()
133             tail = "\n".join(lines[-30:])
134             raise RuntimeError(f"[{label}]运行失败。日志: {log_path}\n{tail}") from exc
135     elapsed = perf_counter() - started
136     print(f"[{label}] 完成, 用时 {elapsed:.1f} s; 日志: {log_path.name}",
flush=True)
137     return elapsed
138
139
140 def write_manifest(output: Path, timings: list[tuple[str, str, float]]) ->
None:
141     missing = [relative for relative in EXPECTED_PNG if not (output /
relative).is_file()]
142     manifest_path = output / "figure_manifest.csv"
143     manifest_path.parent.mkdir(parents=True, exist_ok=True)
144     with manifest_path.open("w", encoding="utf-8-sig", newline="") as
handle:
145         writer = csv.writer(handle)
146         writer.writerow(["relative_path", "exists", "size_bytes"])
147         for relative in EXPECTED_PNG:
148             path = output / relative
149             writer.writerow([relative, path.is_file(), path.stat().st_size
if path.is_file() else 0])
150
151     timing_path = output / "run_timings.csv"
152     with timing_path.open("w", encoding="utf-8-sig", newline="") as handle:
153         writer = csv.writer(handle)
154         writer.writerow(["analysis", "label", "elapsed_seconds"])
155         writer.writerows(timings)
156
157     if missing:
158         formatted = "\n".join(f"- {item}" for item in missing)
159         raise RuntimeError(f"运行完成, 但以下图片缺失: \n{formatted}")
160
161
162 def main() -> None:

```

```

163     args = build_parser().parse_args()
164     if not PACKAGE_DIR.is_dir():
165         raise FileNotFoundError(f"未找到 BatteryModel 包目录: {PACKAGE_DIR}")
166
167     output = Path(args.output).expanduser().resolve()
168     output.mkdir(parents=True, exist_ok=True)
169     mc_runs = args.mc_runs if args.mc_runs is not None else (30 if args.
profile == "report" else 500)
170     if mc_runs < 2:
171         raise ValueError("蒙特卡洛次数至少为 2")
172
173     selected = [(name, label) for name, label in STEPS if args.only in (
None, name)]
174     timings: list[tuple[str, str, float]] = []
175     total_start = perf_counter()
176     print("=" * 72)
177     print("电池 TTE 报告图表一键生成")
178     print(f"运行配置: {args.profile}")
179     print(f"输出目录: {output}")
180     print(f"蒙特卡洛次数: {mc_runs}")
181     print("=" * 72, flush=True)
182
183     for name, label in selected:
184         elapsed = run_step(name, label, output, args.profile, mc_runs)
185         timings.append((name, label, elapsed))
186
187     if args.only is None:
188         write_manifest(output, timings)
189         print(f"\n全部 {len(EXPECTED_PNG)} 张图片均已生成并通过完整性检查。",
flush=True)
190     else:
191         print("\n指定模块运行完成。", flush=True)
192     print(f"总用时: {perf_counter() - total_start:.1f} s")
193     print(f"结果位置: {output}")
194
195
196 if __name__ == "__main__":
197     main()

```